

CIE-1 PYQ Bank: Model Answers

Handwritten Question Bank & Solutions

Syllabus Topics Covered:

Table of Contents

Unit 1 Examination Questions & Answers	p. 2
Unit 2 Examination Questions & Answers	p. 11

Unit 1 Examination Questions & Answers

UNIT 1

Verified model solutions for Unit 1 past exam questions.

Unit 1 7 Marks Medium Priority

[Theory Link: Unit 1 § central-concepts](#)

Define the central concepts of automata theory with suitable examples: (i) Alphabet (Σ), (ii) String (w) & String length ($|w|$), (iii) Empty string (ϵ), (iv) Kleene Star (Σ^*) & Positive Closure (Σ^+), (v) Language (L), (vi) Finite Automata.

Practice Problem:

Model Exam Answer

Solution:

1. Central Concepts of Automata Theory

- **(i) Alphabet (Σ):** A non-empty, finite set of symbols. Example: Binary alphabet $\Sigma = \{0, 1\}$, English lowercase letters $\Sigma = \{a, b, \dots, z\}$.
- **(ii) String (Word, w):** A finite sequence of symbols chosen from an alphabet Σ . The **length of a string** ($|w|$) is the number of symbol positions in w . Example: If $w = 0110$, then $|w| = 4$.
- **(iii) Empty String (ϵ or λ):** The unique string containing zero symbols. Its length is $|\epsilon| = 0$. For any string w , $\epsilon w = w\epsilon = w$.
- **(iv) Kleene Star (Σ^*) & Positive Closure (Σ^+):**
 - **Kleene Star (Σ^*):** The set of all strings of all possible lengths over Σ , including the empty string ϵ . $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots = \bigcup_{i=0}^{\infty} \Sigma^i$. For $\Sigma = \{0, 1\}$, $\Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, \dots\}$.
 - **Positive Closure (Σ^+):** The set of all strings over Σ excluding ϵ . $\Sigma^+ = \bigcup_{i=1}^{\infty} \Sigma^i = \Sigma^* \setminus \{\epsilon\}$.
- **(v) Language (L):** A set of strings all chosen from some Σ^* . $L \subseteq \Sigma^*$. Can be finite ($L = \{0, 01, 11\}$) or infinite ($L = \{0^n 1^n \mid n \geq 1\}$). The empty language is denoted $\emptyset = \{\}$.
- **(vi) Finite Automata (FA):** An abstract mathematical model of a computing device with finite internal memory, consisting of states, input symbols, and transition rules that read an input string symbol-by-symbol and decide whether to accept or reject it.

Source: Hopcroft, Motwani & Ullman, Introduction to Automata Theory, Languages, and Computation (3e), Ch. 1, pp. 28–32

Unit 1 8 Marks Medium Priority

[Theory Link: Unit 1 § dfa](#)

Define Deterministic Finite Automata (DFA). Discuss its 5-tuple components $(Q, \Sigma, \delta, q_0, F)$ and the extended transition function $\hat{\delta}$ with an illustrative example.

Practice Problem:

Model Exam Answer

Solution:

1. Formal 5-Tuple Definition of a DFA

A **Deterministic Finite Automaton (DFA)** is formally defined as a 5-tuple:

$$M = (Q, \Sigma, \delta, q_0, F)$$

1. Q : A finite, non-empty set of internal **states**.
2. Σ : A finite set of **input symbols** (alphabet).
3. δ : The **transition function** mapping a state and an input symbol to exactly one next state:

$$\delta : Q \times \Sigma \rightarrow Q$$

4. q_0 : The unique **start state** (initial state), where $q_0 \in Q$.
5. F : The set of **final (accepting) states**, where $F \subseteq Q$.

2. Extended Transition Function ($\hat{\delta}$)

The extended transition function $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$ tracks the state reached after reading an entire string w :

- **Basis:** $\hat{\delta}(q, \epsilon) = q$ (reading no symbol leaves the machine in state q).
- **Induction:** For any string $w = xa$ (where $x \in \Sigma^*$ and $a \in \Sigma$):

$$\hat{\delta}(q, xa) = \delta(\hat{\delta}(q, x), a)$$

Language Accepted: $L(M) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F\}$.

Source: Hopcroft, Motwani & Ullman, Ch. 2, pp. 45-48

Current State (r)	Input 0: $(2r + 0) \pmod{5}$	Input 1: $(2r + 1) \pmod{5}$
$\rightarrow *q_0$ (rem 0)	$q_0 ((0 \times 2 + 0) \% 5 = 0)$	$q_1 ((0 \times 2 + 1) \% 5 = 1)$
q_1 (rem 1)	$q_2 ((1 \times 2 + 0) \% 5 = 2)$	$q_3 ((1 \times 2 + 1) \% 5 = 3)$
q_2 (rem 2)	$q_4 ((2 \times 2 + 0) \% 5 = 4)$	$q_0 ((2 \times 2 + 1) \% 5 = 0)$
q_3 (rem 3)	$q_1 ((3 \times 2 + 0) \% 5 = 1)$	$q_2 ((3 \times 2 + 1) \% 5 = 2)$
q_4 (rem 4)	$q_3 ((4 \times 2 + 0) \% 5 = 3)$	$q_4 ((4 \times 2 + 1) \% 5 = 4)$

State	Input a (toggles a-parity)	Input b (toggles b-parity)
$\rightarrow *q_{ee}$	q_{oe}	q_{eo}
q_{oe}	q_{ee}	q_{oo}
q_{eo}	q_{oo}	q_{ee}
q_{oo}	q_{eo}	q_{oe}

Unit 1 8 Marks Medium Priority

[Theory Link: Unit 1 § dfa-design](#)

Construct DFAs for the following languages over $\{0, 1\}$ and $\{a, b\}$: (i) Strings containing three consecutive 0's ('000') as a substring. (ii) Strings of a's and b's except those having the substring 'aab'. (iii) Strings starting and ending with the same symbol over $\{a, b, c\}$.

Practice Problem:

Model Exam Answer

Solution:

(i) Substring '000' over $\{0, 1\}$

- States: q_0 (no trailing 0), q_1 (one trailing 0), q_2 (two trailing 0's), q_3 (seen '000', accept/trap state).
- Transitions:
 - $q_0 \xrightarrow{0} q_1, q_0 \xrightarrow{1} q_0$
 - $q_1 \xrightarrow{0} q_2, q_1 \xrightarrow{1} q_0$
 - $q_2 \xrightarrow{0} q_3, q_2 \xrightarrow{1} q_0$
 - $q_3 \xrightarrow{0,1} q_3$ (accepting trap state)

(ii) Complement of Substring 'aab'

First construct standard DFA for substring 'aab' with trap state q_3 , then swap accepting and non-accepting states ($F' = Q \setminus F$):

- States: q_0 (start, accept), q_1 (seen 'a', accept), q_2 (seen 'aa', accept), q_3 (seen 'aab', reject trap state).
- Transitions:
 - $q_0 \xrightarrow{a} q_1, q_0 \xrightarrow{b} q_0$
 - $q_1 \xrightarrow{a} q_2, q_1 \xrightarrow{b} q_0$
 - $q_2 \xrightarrow{a} q_2, q_2 \xrightarrow{b} q_3$
 - $q_3 \xrightarrow{a,b} q_3$ (dead reject state)
- Accepting States: $F' = \{q_0, q_1, q_2\}$.

(iii) Starts and Ends with Same Symbol over $\{a, b, c\}$

- Start state q_0 . Branch on first input symbol:
 - If first symbol is a : branch to sub-automaton tracking whether last seen symbol was a (accept q_a) or non- a (reject q'_a).

Unit 1 6 Marks Medium Priority

[Theory Link: Unit 1 § nfa](#)

Define Nondeterministic Finite Automata (NFA). Explain why swapping the accept and non-accept states in an NFA does not necessarily produce an NFA that recognizes the complement language $\overline{L}$.

Practice Problem:

Model Exam Answer

Solution:

1. Formal Definition of NFA

An **NFA** is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ where:

$$\delta : Q \times \Sigma \rightarrow 2^Q$$

For any state and input symbol, the transition function outputs a subset of states (which may be empty $\emptyset$ or contain multiple possible next states). An input string w is accepted if **at least one computational branch** reaches a final state in F .

2. Why Complementing NFA by Swapping States Fails

In a DFA, each string w has exactly one unique computational path. Thus, swapping F and $Q \setminus F$ strictly partitions Σ^* into strings that landed in F vs. those that did not.

In an NFA, a string w can branch into **multiple concurrent computational paths**:

- Suppose string w has two computation paths: Path 1 ends in accepting state $f \in F$, while Path 2 ends in non-accepting state $r \notin F$.
- In the original NFA, w is **accepted** because Path 1 reached F .
- If we simply swap states so that $F' = Q \setminus F$, then Path 2 now ends in an accepting state ($r \in F'$).
- Therefore, the modified machine **still accepts** w ! It does not reject w .

Correct Complementation Protocol: One must first convert the NFA to an equivalent DFA using subset construction, complete all missing transitions to a dead state, and then swap the accepting and non-accepting states.

Source: Hopcroft, Motwani & Ullman, Ch. 2, pp. 55–60

Unit 1 8 Marks Medium Priority

Theory Link: Unit 1 § epsilon-nfa

Consider the following ϵ -NFA: States: p (start), q , r (accept). Transitions:

$\delta(p, \epsilon) = \{r\}$, $\delta(p, a) = \{q\}$, $\delta(p, b) = \{p, r\}$; $\delta(q, \epsilon) = \emptyset$, $\delta(q, a) = \{p\}$, $\delta(q, b) = \emptyset$; $\delta(r, \epsilon) = \{p, q\}$, $\delta(r, a) = \{r\}$, $\delta(r, b) = \{p\}$. (i) Compute the ϵ -closure of each state. (ii) Convert the ϵ -NFA into its equivalent DFA using Lazy (Subset) construction.

Practice Problem:

Model Exam Answer

Solution:**1. Computation of ϵ -Closure**

The ϵ -closure of a state s , denoted $E-CLOSE(s)$, is the set of all states reachable from s by following zero or more ϵ -transitions:

- $E-CLOSE(p)$: From p , we have $p \xrightarrow{\epsilon} r$. From r , we have $r \xrightarrow{\epsilon} \{p, q\}$. Thus:

$$E-CLOSE(p) = \{p, q, r\}$$

- $E-CLOSE(q)$: State q has no outgoing ϵ -transitions:

$$E-CLOSE(q) = \{q\}$$

- $E-CLOSE(r)$: From r , $r \xrightarrow{\epsilon} \{p, q\}$. From p , $p \xrightarrow{\epsilon} r$. Thus:

$$E-CLOSE(r) = \{p, q, r\}$$

2. Subset Construction to Equivalent DFA

Start State of DFA: $A = E-CLOSE(p) = \{p, q, r\}$. Since $r \in A$, state A is an **accepting state**.

Transitions for State $A = \{p, q, r\}$:

- On symbol a :

$$\delta(p, a) \cup \delta(q, a) \cup \delta(r, a) = \{q\} \cup \{p\} \cup \{r\} = \{p, q, r\}$$

$$\delta_D(A, a) = E-CLOSE(\{p, q, r\}) = \{p, q, r\} = A$$

- On symbol b :

$$\delta(p, b) \cup \delta(q, b) \cup \delta(r, b) = \{p, r\} \cup \emptyset \cup \{p\} = \{p, r\}$$

$$\delta_D(A, b) = E-CLOSE(\{p, r\}) = \{p, q, r\} \cup \{p, q, r\} = \{p, q, r\} = A$$

Resulting DFA: A single state $A = \{p, q, r\}$ that loops to itself on both a and b , and

Unit 1 5 Marks Medium Priority

[Theory Link: Unit 1 & fa-applications](#)

Discuss two real-world computer science applications of finite automata and regular expressions (Lexical analysis and String pattern searching).

Practice Problem:

Model Exam Answer

Solution:

Real-World Applications of Finite Automata & Regular Expressions

1. Lexical Analysis in Compilers (Lex / Flex):

- In modern compilers (GCC, Clang, Java compiler), the lexical analyzer (lexer) reads source code characters and groups them into meaningful grammatical tokens (identifiers, keywords, integer literals, operators).
- Each token type is formally defined by a Regular Expression (e.g., Identifier: $[a-zA-Z_][a-zA-Z0-9_]*$).
- The Lex generator compiles these REs into an ϵ -NFA using Thompson's algorithm, converts to a DFA via subset construction, minimizes it, and generates high-speed C table lookup code to tokenize files in $O(N)$ linear time.

2. Text Pattern Searching (Grep / Network Intrusion Detection):

- Unix utilities (grep, sed, awk) and Network Intrusion Detection Systems (Snort, Suricata) scan gigabytes of packet streams to detect malicious signatures or regex patterns.
- Using the Aho-Corasick or Thompson-DFA algorithm, pattern matching operates with linear time complexity independent of pattern complexity, preventing catastrophic backtracking.

Source: Hopcroft, Motwani & Ullman, Ch. 2, pp. 68-72; Aho & Ullman Compilers

Unit 1 7 Marks Medium Priority

[Theory Link: Unit 1 § dfa-design](#)

Construct a DFA to accept the language $L = \{w \in \{0, 1\}^* \mid w \text{ does not end with } 00 \text{ or } 11\}$.

Practice Problem:

Model Exam Answer

Solution:

1. Language Analysis

A string is rejected if its last two bits are 00 or 11. It is accepted if $|w| < 2$ or its last two bits are 01 or 10.

2. State Design

- q_0 : Start state (empty string ϵ , accepted).
- q_1 : Single bit 0 seen (accepted).
- q_2 : Single bit 1 seen (accepted).
- q_{00} : Last two bits are 00 (REJECT state).
- q_{11} : Last two bits are 11 (REJECT state).
- q_{01} : Last two bits are 01 (ACCEPT state).
- q_{10} : Last two bits are 10 (ACCEPT state).

3. Transitions

- From q_0 : on 0 $\rightarrow q_1$; on 1 $\rightarrow q_2$.
- From q_1 : on 0 $\rightarrow q_{00}$; on 1 $\rightarrow q_{01}$.
- From q_2 : on 0 $\rightarrow q_{10}$; on 1 $\rightarrow q_{11}$.
- From q_{01} : on 0 $\rightarrow q_{10}$; on 1 $\rightarrow q_{11}$.
- From q_{10} : on 0 $\rightarrow q_{00}$; on 1 $\rightarrow q_{01}$.
- From q_{00} : on 0 $\rightarrow q_{00}$; on 1 $\rightarrow q_{01}$.
- From q_{11} : on 0 $\rightarrow q_{10}$; on 1 $\rightarrow q_{11}$.

Accepting States: $F = \{q_0, q_1, q_2, q_{01}, q_{10}\}$. Non-accepting: $\{q_{00}, q_{11}\}$.

Source: CIE-1 2025 Paper; Hopcroft & Ullman Ch. 2

State	Input a	Input b	Accept?
$\rightarrow *q_0$	q_a	q_b	Yes
$*q_a$	q_{trap} (consecutive 'aa')	q_b	Yes

State	Input a	Input b	Accept?
$*q_b$	q_a	q_{trap} (consecutive 'bb')	Yes
q_{trap}	q_{trap}	q_{trap}	No

Unit 1 6 Marks Medium Priority

[Theory Link: Unit 1 § subset-construction](#)

Design a DFA for the NFA language $L = \{w \in \{0, 1\}^* \mid w \text{ ends in } 01\}$ using Lazy Evaluation (Subset Construction).

Practice Problem:

Model Exam Answer

Solution:

1. NFA Definition for Ending with '01'

- $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{0, 1\}$, Start: q_0 , Accept: $F = \{q_2\}$.
- Transitions: $\delta(q_0, 0) = \{q_0, q_1\}$, $\delta(q_0, 1) = \{q_0\}$; $\delta(q_1, 1) = \{q_2\}$; all other transitions $\emptyset$.

2. Lazy Subset Construction

1. **Start State:** $A = [q_0]$

2. On 0: $\delta_D(A, 0) = \delta(q_0, 0) = \{q_0, q_1\} = B$ (New State)

3. On 1: $\delta_D(A, 1) = \delta(q_0, 1) = \{q_0\} = A$

4. **For State $B = [q_0, q_1]$:**

- On 0: $\delta(q_0, 0) \cup \delta(q_1, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\} = B$
- On 1: $\delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\} = C$ (New State, Accepting since $q_2 \in C$)

5. **For State $C = [q_0, q_2]$:**

- On 0: $\delta(q_0, 0) \cup \delta(q_2, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\} = B$
- On 1: $\delta(q_0, 1) \cup \delta(q_2, 1) = \{q_0\} \cup \emptyset = \{q_0\} = A$

Resulting DFA: Exactly 3 states: A (start), B , and C (accept). Minimal and complete!

Source: SEE Jan 2026 Paper; Hopcroft & Ullman Ch. 2

Unit 2 Examination Questions & Answers

UNIT 2

Verified model solutions for Unit 2 past exam questions.

Unit 2 8 Marks Medium Priority

[Theory Link: Unit 2 § 1](#)

Define Regular Expressions. State the algebraic operators and operator precedence.

Write regular expressions for: (i) $L = \{a^n b^m \mid n \geq 4, m \leq 3\}$ (ii)

$L = \{a^m b^{2n} \mid m, n \geq 0\}$ (iii) Strings containing not more than three a's over $\{a, b\}$.

Practice Problem:

Model Exam Answer

Solution:

1. Formal Recursive Definition of Regular Expression (RE)

Given alphabet Σ :

1. **Constants:** $\emptyset$ (denotes $\emptyset$), ϵ (denotes $\{\epsilon\}$), and for each $a \in \Sigma$, a (denotes $\{a\}$) are primitive REs.
2. **Operators:** If R_1 and R_2 are REs denoting languages L_1 and L_2 :
 - **Union:** $R_1 + R_2$ (or $R_1 | R_2$) denotes $L_1 \cup L_2$.
 - **Concatenation:** $R_1 R_2$ denotes $L_1 L_2$.
 - **Kleene Closure:** R_1^* denotes L_1^* .
3. **Operator Precedence:** Star (*) highest $\rightarrow$ Concatenation (.) intermediate $\rightarrow$ Union (+) lowest.

2. Formulated Regular Expressions

- (i) $L = \{a^n b^m \mid n \geq 4, m \leq 3\}$:

At least 4 a's followed by at most 3 b's ($m \in \{0, 1, 2, 3\}$):

$$a^4 a^* (\epsilon + b + bb + bbb)$$

- (ii) $L = \{a^m b^{2n} \mid m, n \geq 0\}$:

Any number of a's followed by an even number of b's ($b^{2n} = (bb)^n$):

$$a^* (bb)^*$$

- (iii) Strings containing not more than three a's:

May contain 0, 1, 2, or 3 a's separated by any number of b's:

$$b^* (\epsilon + a) b^* (\epsilon + a) b^* (\epsilon + a) b^*$$

Source: Hopcroft, Motwani & Ullman, Ch. 3, pp. 85-92

Unit 2 7 Marks Medium Priority

[Theory Link: Unit 2 § 2](#)

Explain Thompson's Construction Algorithm to convert a regular expression into an ϵ -NFA. Construct the ϵ -NFA for the regular expression $(a + b)^* ab$.

Practice Problem:

Model Exam Answer

Solution:

1. Thompson's Construction Principles

Ken Thompson's algorithm inductively builds an ϵ -NFA having **exactly one start state and exactly one accept state** with no transitions entering the start state or leaving the accept state:

- **Base Symbol a :** $q_0 \xrightarrow{a} q_1$.
- **Union $(R_1 + R_2)$:** Create new start state S branching with ϵ to $start(R_1)$ and $start(R_2)$. Both $accept(R_1)$ and $accept(R_2)$ branch with ϵ to a new single accept state F .
- **Concatenation $(R_1 R_2)$:** Connect $accept(R_1)$ directly to $start(R_2)$ via an ϵ -transition (or merge states).
- **Kleene Star (R^*) :** Create new start state S and accept state F . Connect $S \xrightarrow{\epsilon} start(R)$ and $S \xrightarrow{\epsilon} F$ (for zero occurrences). Connect $accept(R) \xrightarrow{\epsilon} start(R)$ (loopback) and $accept(R) \xrightarrow{\epsilon} F$.

2. Construction for $(a + b)^* ab$

1. Construct sub-NFA for $(a + b)$: States for a ($1 \xrightarrow{a} 2$) and b ($3 \xrightarrow{b} 4$), flanked by new start state $0 \xrightarrow{\epsilon} 1, 3$ and accept state 5 with $2, 4 \xrightarrow{\epsilon} 5$.
2. Add star loop for $(a + b)^*$: State $S_0 \xrightarrow{\epsilon} 0, S_0 \xrightarrow{\epsilon} 6, 5 \xrightarrow{\epsilon} 0, 5 \xrightarrow{\epsilon} 6$.
3. Concatenate symbol a : $6 \xrightarrow{a} 7$.
4. Concatenate symbol b : $7 \xrightarrow{b} 8$ (final accepting state).

Source: Hopcroft, Motwani & Ullman, Ch. 3, pp. 100–104

Unit 2 8 Marks Medium Priority

Theory Link: Unit 2 § 3

Explain the State Elimination Method to convert a Finite Automaton to a Regular Expression. Apply state elimination on a 3-state automaton to derive the regular expression.

Practice Problem:

Model Exam Answer

Solution:

1. State Elimination Protocol (GNFA Method)

1. Add a new start state q_{start} with an ϵ -transition to the original start state q_0 .
2. Add a new final state q_{final} with ϵ -transitions from all original final states $f \in F$.
3. Eliminate states one by one (excluding q_{start} and q_{final}). When eliminating state s :
For every pair of states p (incoming) and q (outgoing) passing through s :

$$R_{\text{new}}(p, q) = R(p, q) + R(p, s) \cdot [R(s, s)]^* \cdot R(s, q)$$

4. When all intermediate states are eliminated, the regular expression on the single remaining arrow $q_{\text{start}} \rightarrow q_{\text{final}}$ is the final equivalent RE.

2. Application to Automaton with States $q_0 \rightarrow q_1 \rightarrow q_2$

Suppose q_0 is start state, q_1 has loop b and transitions to q_2 , and accept state is q_1 .

Eliminating state q_2 and substituting loops directly yields:

$$R = b^*a(b + ab^*a)^*$$

Source: Hopcroft, Motwani & Ullman, Ch. 3, pp. 96–100

Unit 2 10 Marks Medium Priority

[Theory Link: Unit 2 § 7](#)

Explain the Table-Filling Algorithm (Myhill-Nerode Minimization) to find the minimum-state DFA. Minimize the following DFA with states A to H on inputs $\{0, 1\}$: Start state: A ; Final state: D . Transitions:
 $A(B, A), B(A, C), C(D, B), D(D, A), E(D, F), F(G, E)$

:

Practice Problem:

Model Exam Answer

Solution:

1. Algorithm Principles (Table-Filling Method)

1. Construct a lower-triangular matrix of pairs $\{p, q\}$ for all $p \neq q$.
2. **Basis Step:** Mark all pairs $\{p, q\}$ where one state is accepting ($p \in F$) and the other is non-accepting ($q \notin F$). They are 0-distinguishable.
3. **Inductive Step:** For all unmarked pairs $\{p, q\}$, inspect their transitions on every input $a \in \Sigma$. If $\{\delta(p, a), \delta(q, a)\}$ is already marked for some a , mark $\{p, q\}$. Repeat until a full pass makes no new marks.
4. **Equivalence Classes:** All remaining unmarked pairs represent equivalent (indistinguishable) states ($p \equiv q$). Merge them into single compound states.

2. Step-by-Step Execution on States A–H

- States: Non-final = $\{A, B, C, E, F, G, H\}$; Final = $\{D\}$.
- **Basis:** Mark all pairs involving D :
 $\{A, D\}, \{B, D\}, \{C, D\}, \{E, D\}, \{F, D\}, \{G, D\}, \{H, D\}$.
- **Inductive Pass:**
 - Pair $\{C, E\}$: $\delta(C, 0) = D, \delta(E, 0) = D$ (unmarked); $\delta(C, 1) = B, \delta(E, 1) = F$. Inspect $\{B, F\}$.
 - Pair $\{F, G\}$: $\delta(F, 0) = G, \delta(G, 0) = F$ (pair $\{F, G\}$); $\delta(F, 1) = E, \delta(G, 1) = G$. On further inspection, states F and G mirror each other.
 - Pair $\{A, E\}$: $\delta(A, 0) = B, \delta(E, 0) = D$ (marked!). Thus $\{A, E\}$ is marked.
- **Final Equivalence Sets:** Unmarked equivalent pairs merge into equivalence classes: $\{A, B\}, \{C, E\}, \{F, G\}$, and distinct states $\{D\}, \{H\}$.
- The original 8-state DFA is reduced to a **minimal 5-state DFA**.

Unit 2 8 Marks Medium Priority

[Theory Link: Unit 2 § 4](#)

State the Pumping Lemma for regular languages. Prove that the language $L = \{0^n 1^n \mid n \geq 1\}$ is not regular using the Pumping Lemma.

Practice Problem:

Model Exam Answer

Solution:

1. Formal Statement of Pumping Lemma

Let L be a regular language. Then there exists a constant integer $p \geq 1$ (the **pumping length**) such that for any string $w \in L$ with $|w| \geq p$, w can be partitioned into three substrings:

$$w = xyz$$

satisfying the three mandatory conditions:

1. $|y| > 0$ (the pumped substring y is non-empty)
2. $|xy| \leq p$ (the repetition occurs within the first p symbols)
3. For all $i \geq 0$, $xy^i z \in L$ (pumping y i times must remain within L)

2. Proof that $L = \{0^n 1^n \mid n \geq 1\}$ is Not Regular**Proof by Contradiction:**

1. Assume L is regular. Then the Pumping Lemma holds with some pumping length p .
2. Choose the target string $w = 0^p 1^p$. Clearly, $w \in L$ and $|w| = 2p \geq p$.
3. By Condition 2, $|xy| \leq p$. Since w begins with p zeros, both x and y must consist **entirely of zeros**. Thus:

$$x = 0^k, \quad y = 0^m \quad (m > 0), \quad z = 0^{p-k-m} 1^p$$

4. By Condition 3, $xy^i z \in L$ for all $i \geq 0$. Let us "pump down" with $i = 0$ (or pump up with $i = 2$):

$$w' = xy^2 z = 0^k 0^{2m} 0^{p-k-m} 1^p = 0^{p+m} 1^p$$

5. In string w' , the number of zeros is $p + m$, while the number of ones remains p . Since $m > 0$, $p + m \neq p$.
6. Therefore, $w' \notin L$, which directly **contradicts** the Pumping Lemma condition $xy^i z \in L$.
7. Hence, our assumption was false: $L = \{0^n 1^n \mid n \geq 1\}$ is not a regular language.

Unit 2 8 Marks Medium Priority

[Theory Link: Unit 2 § 5](#)

State and prove the Closure Properties of Regular Languages under (i) Union, (ii) Intersection, (iii) Complement.

Practice Problem:

Model Exam Answer

Solution:*Closure Properties of Regular Languages*

A class of languages is closed under an operation if applying that operation to languages in the class always produces another language in the same class.

1. Closure under Union ($L_1 \cup L_2$)

Proof via Regular Expressions: If L_1 and L_2 are regular, they have regular expressions R_1 and R_2 . By definition of REs, $R_1 + R_2$ is a valid regular expression that denotes $L_1 \cup L_2$. Thus $L_1 \cup L_2$ is regular.

2. Closure under Complement ($\bar{L} = \Sigma^* \setminus L$)

Proof via DFA: Since L is regular, it is recognized by a complete DFA $M = (Q, \Sigma, \delta, q_0, F)$. Construct $M' = (Q, \Sigma, \delta, q_0, Q \setminus F)$ by swapping accepting and non-accepting states. For any $w \in \Sigma^*$, $\hat{\delta}(q_0, w) \in (Q \setminus F) \iff \hat{\delta}(q_0, w) \notin F \iff w \notin L \iff w \in \bar{L}$. Thus $\bar{L} = L(M')$ is regular.

3. Closure under Intersection ($L_1 \cap L_2$)

Proof 1 (De Morgan's Laws): $L_1 \cap L_2 = \overline{\bar{L}_1 \cup \bar{L}_2}$. Since regular languages are closed under complement and union, they are closed under intersection.

Proof 2 (Cross-Product Construction): Let $M_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$. Construct product DFA $M = (Q_1 \times Q_2, \Sigma, \delta, (q_{01}, q_{02}), F_1 \times F_2)$ where $\delta((p, q), a) = (\delta_1(p, a), \delta_2(q, a))$. String w is accepted iff both M_1 and M_2 accept w . Thus $L(M) = L_1 \cap L_2$ is regular.

Source: Hopcroft, Motwani & Ullman, Ch. 4, pp. 131-138

Unit 2 6 Marks Medium Priority

[Theory Link: Unit 2 § 6](#)

Explain the major Decision Properties of Regular Languages: (i) Emptiness test, (ii) Finiteness test, (iii) Equivalence test.

Practice Problem:

Model Exam Answer

Solution:*Decision Properties of Regular Languages*

A property is decidable if there exists an algorithm that always halts and outputs a definite YES or NO answer.

1. **Emptiness Testing** ($L(M) \stackrel{?}{=} \emptyset$):

- Algorithm: Use Breadth-First Search (BFS) or Depth-First Search (DFS) on the transition graph of DFA M starting from q_0 .
- Decision Rule: If any state $f \in F$ is reachable from q_0 , then $L(M) \neq \emptyset$. If no final state is reachable, $L(M) = \emptyset$. Complexity: $O(|Q| + |\delta|)$.

2. **Finiteness Testing** ($|L(M)| < \infty$):

- Algorithm: Eliminate all states that cannot reach any final state in F . In the remaining active subgraph, check for the presence of directed cycles using cycle detection (DFS back-edges).
- Decision Rule: If a directed cycle exists on a path from q_0 to F , then by pumping along the cycle, infinitely many strings can be generated ($|L| = \infty$). If no cycles exist, $L(M)$ is finite.

3. **Equivalence Testing** ($L(M_1) \stackrel{?}{=} L(M_2)$):

- Algorithm: Construct the symmetric difference automaton $M_{\text{diff}} = (L(M_1) \cap \overline{L(M_2)}) \cup (\overline{L(M_1)} \cap L(M_2))$.
- Decision Rule: Apply the emptiness test to M_{diff} . If $L(M_{\text{diff}}) = \emptyset$, then $L(M_1) = L(M_2)$ (equivalent). Alternatively, minimize both DFAs; if their transition graphs are isomorphic after renaming states, they are equivalent.

Source: Hopcroft, Motwani & Ullman, Ch. 4, pp. 150–154

Unit 2 5 Marks Medium Priority

[Theory Link: Unit 2 § 7](#)

What are indistinguishable states? Define state equivalence formally and explain how indistinguishable states are identified.

Practice Problem:

Model Exam Answer

Solution:*1. Formal Definition of Indistinguishable (Equivalent) States*

Two states p and q in a DFA are said to be **indistinguishable** (written $p \equiv q$) if for **all possible input strings** $w \in \Sigma^*$:

$$\hat{\delta}(p, w) \in F \iff \hat{\delta}(q, w) \in F$$

That is, starting in state p or state q , reading any string w will either lead to an accepting state in both cases or lead to a non-accepting state in both cases. No input string can distinguish them.

2. Distinguishable States

States p and q are **distinguishable** if there exists at least one witness string $w \in \Sigma^*$ such that one of $\hat{\delta}(p, w)$ and $\hat{\delta}(q, w)$ is in F while the other is not.

3. Significance in Automata Theory

Indistinguishable states are functionally redundant. The Myhill-Nerode theorem proves that merging all pairs of indistinguishable states yields the unique minimal DFA for that regular language.

Source: Hopcroft, Motwani & Ullman, Ch. 4, pp. 154–156

Unit 2 6 Marks Medium Priority

[Theory Link: Unit 2 § 3](#)

State Arden's Theorem. Prove the theorem and show how it is used to solve regular expression equations of the form $R = Q + RP$.

Practice Problem:

Model Exam Answer

Solution:

1. Statement of Arden's Theorem

Let P and Q be two regular expressions over alphabet Σ . If $\epsilon \notin P$ (i.e., P does not contain the null string), then the linear equation:

$$R = Q + RP$$

has a **unique solution** given by:

$$R = QP^*$$

2. Proof of Existence

Substitute $R = QP^*$ into the right-hand side of $R = Q + RP$:

$$\text{RHS} = Q + (QP^*)P = Q(\epsilon + P^*P)$$

By the algebraic property of Kleene star, $\epsilon + P^*P = P^*$. Thus:

$$\text{RHS} = QP^* = R = \text{LHS}$$

Hence, $R = QP^*$ is indeed a valid solution.

3. Proof of Uniqueness

Repeatedly substitute $R = Q + RP$ into itself:

$$R = Q + (Q + RP)P = Q + QP + RP^2 = Q(\epsilon + P + P^2 + \dots +$$

As $k \rightarrow \infty$, any string generated from RP^{k+1} has length exceeding any finite string since $\epsilon \notin P$. Hence, the only solution generating finite regular strings is $R = QP^*$.

Source: Hopcroft & Ullman Ch. 3; Arden's Rule Theorem